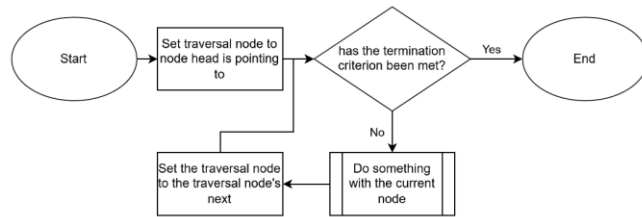


LINKED LISTS – COMMON ALGORITHMS.

TRAVERSAL



Traversal stopping conditions:

- `node == 0` OR `node->next == 0`
- `node->data != data`
- `node->next < or > data` if list is sorted

INSERTION

- Always create a `newNode` and set `newNode->next` and/or `newNode->prev` to `NULL` (if constructor isn't given)

Cases for insertion:

- Insert into empty list.
- Prepend (Front/Head)
- Append (Rear/Tail)
- Insert into the middle.
- Always sketch it to get an idea of order of updating pointers!
- A `prevNode` pointer might be necessary.

DELETION

- Always make a `tempNode` pointer to the node you found and disconnect `currentNode` from the list before deleting.

Cases for deletion:

- Delete from empty list.
- Delete from start of list.
- Delete from end of list (`node->next == 0`)
- Delete from middle of list.
- Not deleting (Value not found)
- Always sketch it to get an idea of order of updating pointers!
- A `prevNode` pointer might be necessary.

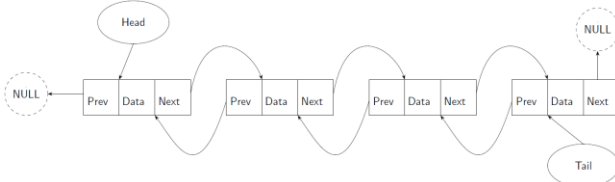
GENERAL

- Check for `NULL` everywhere!
- Always apply the above-mentioned edge cases

LINKED LISTS – VARIANTS.

DOUBLY LINKED LIST

General structure:



Important condition:

- A doubly linked list is empty if and only if `head == 0`.

INSERTION

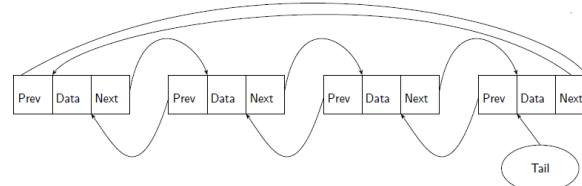
- Insertion algorithm does not change from the general linked list insertion; same cases apply.
- Relinking changes: updating `tail` and `tail->next` may be necessary if the list has a tail. Update `prev`!
- Sketch to get the order of operations!

DELETION

- Deletion algorithm does not change from the general linked list deletion; same cases apply.
- Relinking changes: updating `tail` and `tail->next` may be necessary if the list has a tail. Update `prev`!
- Sketch to get the order of operations!

CIRCULAR LINKED LIST

General structure:



Important conditions:

- A circular linked list is empty if and only if `head == 0`
- Equivalently, it is empty if and only if `tail->next == 0`
- A list has a lone node iff `tail->next == tail`
- Stop condition: `node == head/node == tail->next`
- Traversal block: `do { ... } while(node != head);`

INSERTION

- Insertion algorithm does not change from the general linked list insertion; same cases apply.
- Sketch to get the order of operations!

DELETION

- Deletion algorithm does not change from the general linked list deletion; same cases apply.
- Sketch to get the order of operations!

LINKED LISTS – APPLICATIONS.

STACK

- LIFO structure
- Last inserted is first to be removed.
- Has a `top` member variable which points to where to insert or remove.
- Can only access the `top` node, not any of the others.
- Usual functions:
 - Pop
 - Push
 - Top
 - Is Empty
- Only cases of insertion and deletion:
 - Insertion/Deletion from front of list
 - Insertion/Deletion from empty list

QUEUE

- FIFO structure
- First inserted is first to be removed.
- Has a `front` member variable which points to where data is removed.
- Has a `rear` member variable which points to where data is inserted
- Can only access the `front` node, not any of the others.
- Usual functions:
 - Dequeue
 - Enqueue
 - Front
 - Is empty.
- Cases of insertion:
 - Insertion into empty list
 - Insertion into the rear (no traversal needed!)
- Cases of deletion:
 - Deletion from empty list
 - Deletion from the front

STANDARD TEMPLATE LIBRARY (STL)

- Make sure to `#include <container_name>` in the header.
- Make sure to add `std::` scope resolution

Sequential Containers

Container name	Description	Iterator type
<code>std::vector<T></code>	An expandable array. Values may be added to or removed from the end or middle of a vector .	Random access
<code>std::deque<T></code>	Like a vector , but allows values to be added or removed from front.	Random access
<code>std::list<T></code>	A doubly linked list of data elements. Values may be inserted to or removed from any position.	Bidirectional

Associative Containers

Container name	Description	Duplicates allowed?	Iterator type
<code>std::set<T></code>	Stores a set of keys.	No	Bidirectional
<code>std::multiset<T></code>		Yes	
<code>std::map<K,V></code>	Maps a set of keys to data elements. Only one key per data element is allowed.	No	
<code>std::multimap<K,V></code>	Maps a set of keys to data elements. Many keys per data element are allowed.	Yes	

- Another variation of map is `unordered_map`—efficient because keys aren't sorted.
- Example:
`map<int, string> myMap = { {1, "banana"}, {2, "milk"} };`
- Keys can have gaps between them
- Values do not have to be unique, but keys must be unique.

Iterators

A special kind of pointer used to move through elements in an STL container.

Declaration

```
vector<int> vect;  
for (int i = 1; i < 6; i++) vect.push_back(i);  
  
vector<int>::iterator it; // declare an iterator object  
it = vect.begin(); // assign it to the start of the vector
```

Operations/Functions

Category	Operators / Functions	Meaning	Valid For
Access	*it, it->member	Read/write element it points to	All iterators
Movement (1 step)	++it, it++	Move forward to next element	Most iterators
	--it, it--	Move backward to previous element	Bidirectional & random-access
Movement (n steps)	it + n, it - n, it[n]	Jump ahead/back or index like an array	Random-access (vector, deque)
Comparison	it1 == it2, it1 != it2	Check if two iterators point to same element	All iterators
Positions	begin(), end()	Start and past-the-end positions	All containers
Reverse positions	rbegin(), rend()	End and before-begin (reverse traversal)	Containers supporting reverse iterators

Iterator types

Type	Description
container::iterator	Read/write iterator
container::const_iterator	Read-only iterator

Algorithms

- Important to have these to reduce debugs.
- Make sure to `#include <algorithm>` in the header.
- Make sure to add `std::` scope resolution

Algorithm	Return Type	Description
binary_search(iter1, iter2, value);	bool	Performs a binary search on a sorted range . Returns true if value is found, false otherwise.
count(iter1, iter2, value);	int	Counts the number of times value appears in the range [iter1, iter2).
find(iter1, iter2, value);	iterator	Finds the first element equal to value in [iter1, iter2). Returns iterator to it, or iter2 if not found.
for_each(iter1, iter2, func);	Func (the function object passed)	Calls the function func on each element in [iter1, iter2).
max_element(iter1, iter2);	iterator	Returns an iterator to the largest element in the range [iter1, iter2).
min_element(iter1, iter2);	iterator	Returns an iterator to the smallest element in the range [iter1, iter2).
random_shuffle(iter1, iter2);	void	Randomly reorders the elements in the range [iter1, iter2).
sort(iter1, iter2);	void	Sorts elements in [iter1, iter2) in ascending order .

RECURSION

STUCK IN A PROBLEM? CONSIDER THE FOLLOWING:

1. When should we stop (what is the base case)?
2. How do we breakdown the problem into sub-problems?
3. Does each recursive call move closer to the base case? What must each call do to reach it?
4. What does each call return?
5. Draw the call trace!
6. Try using a helper function!

SAMPLE ALGORITHMS

```
int factorial(int num)
{
    if (num > 1) // recursive case
        return num*factorial(num - 1); // base case
    return 1;
}
```

```
int fib(int n)
{ // return nth Fibonacci number
    if (n == 0 || n == 1) return n; // base case
    return fib(n - 1) + fib(n - 2); //recursive case
}
```

```
void Recursive::printout(const char* str)
{
    if (*str == '\0')
    {
        cout << '\n';
        return;
    }
    cout << *str; //Print then call!
    printout(str + 1);
}
```

```
void printInReverse(const char* str)
{
    if (*str == '\0') return;
    printInReverse(str + 1);
    cout << *str; //Call then print!
}
```

```
int LinkedList::countNodes(ListNode * nodePtr)
{
    if (!nodePtr) return 0;
    return countNodes(nodePtr->next) + 1;
}
```

```
int binarySearch(int array[], int first, int last, int value)
{
    if (first > last) throw "Value not found!";
    int middle = (first + last) / 2; // Midpoint of search
    if (array[middle] == value) return middle;
    if (array[middle] < value)
        return binarySearch(array, middle+1, last, value);
    return binarySearch(array, first, middle-1, value);
}
```

```
void moveDiscs(int num, int fromPeg, int toPeg, int tempPeg)
{ //Towers of Hanoi
    if (num > 0)
    {
        moveDiscs(num - 1, fromPeg, tempPeg, toPeg);
        cout << "Move a disc from peg " << fromPeg
              << " to peg " << toPeg << endl;
        moveDiscs(num - 1, tempPeg, toPeg, fromPeg);
    }
}
```

```
int sum(int arr[], int n)
{
    if (n == 0) return 0; //Base case: no elements
    return arr[n - 1] + sum(arr, n - 1); // Recursive case
}
```

```
void reverseString(string &str, int start, int end)
{
    if (start >= end) return; // Base case
    swap(str[start], str[end]); //include <algorithm>
    reverseString(str, start + 1, end - 1); //Recursive case
}
```

```
bool isPalindrome(const string &str, int start, int end)
{
    if (start >= end) return true; //Base case: finished checking
    if (str[start] != str[end]) return false; // Characters mismatch
    return isPalindrome(str, start + 1, end - 1); //Recurse on inner substring
}
```

MOVING FROM ITERATION TO RECURSION:

1. Identify the loop's purpose (is it summing, printing, searching, etc.?)
2. Determine the base case (usually the loop's termination condition)
3. Perform the work once per call. (Everything in the loop becomes the body of the recursive call.)
4. Call recursively, move toward the base case.

MOVING FROM RECURSION TO ITERATION:

1. Understand: what is the function doing before/after recursion
2. Loop variable replaces recursive parameter
3. If base case has initial value, initialise loop variable to base case value
4. Draw a call stack if you have multiple calls.

ALWAYS CHECK FOR NULLPTR!

RECAP: FRIENDS AND TEMPLATE CLASSES

```
template <class T> class Vector; // forward-declare the class
template <class T> // forward-declare the template function:
ostream& operator<<(ostream& os, Vector<T>& vec);
```

```
template <class T>
class Vector {
public:
    Vector(int);
    T & operator[](int i); // return i'th element of type T
    void print();
    friend ostream& operator<< (<> (ostream& os, Vector<T>& vec);

private:
    T * coords; // array of vector coordinates of type T
    int dimension;
};
```